

SRMS-002 — The Base–Delta Asymmetry Principle

Repository role: This paper establishes the central theoretical principle of *Structural Recognition above Metric Similarity (SRMS)*: the semantic, logical, or runtime importance of a structural delta is not proportional to its geometric size relative to the inherited base representation.

Sizhe Tan

with chatGPT (OpenAI)

2026-07-10

Repository: <https://github.com/sizhet/Structural-Recognition-above-Metric-Similarity>

Abstract

Many intelligent representations are formed through inheritance rather than complete reconstruction.

A new object often reuses a large existing structural base and adds only a small delta:

$$\begin{aligned} &[\\ O' &= B + \Delta \\ &] \end{aligned}$$

The base (B) may contain most of the object's visual, semantic, linguistic, or operational content. The delta (Δ) may consist of only one marker, relation, count increment, role assignment, scope boundary, edge direction, permission state, or trigger condition.

From a geometric perspective, a small delta may contribute little to the distance between (B) and ($B+\Delta$). Their cosine similarity may remain extremely close to one.

From a structural perspective, however, the same delta may change:

- identity;
- cardinality;
- logical polarity;
- argument role;

- relation direction;
- authorization;
- execution order;
- branch membership;
- Runtime Invariant;
- runtime consequence.

This creates a fundamental asymmetry:

```
[
\boxed{
\text{Geometric Delta Size}
\neq
\text{Structural Delta Importance}
}
]
```

This paper names that asymmetry the **Base–Delta Asymmetry Principle**.

The principle explains why holistic metric similarity can perform exceptionally well while still failing on small but decisive structural differences. It also establishes the theoretical basis for Recognition-Gated Cosine Scoring Trees, typed delta registries, hard structural gates, and runtime-aware candidate routing.

1. The Inherited-Base Pattern

Intelligent systems rarely create every new representation from nothing.

They typically reuse a known structure and add a limited modification.

The general pattern is:

```
[
O' = B + \Delta
]
```

where:

- (B) is an inherited structural base;
- (Δ) is a new difference;
- (O') is the resulting object.

Examples include:

cat

cat + instance marker

execute

execute + negation

A calls B

A calls B + conditional trigger

authorized user

authorized user + revoked status

known Chinese character component

known component + semantic radical + phonetic marker

existing UTN node

existing UTN node + new role or edge

This pattern is computationally efficient because the system does not rebuild the entire object.

It reuses what is already known.

That reuse is consistent with the Minimal Evolution Threshold:

```
[  
\text{Preserve the existing structure}  
+  
\text{add the minimum sufficient difference}  
]
```

The resulting delta may be small because the existing base already carries most of the required structure.

But the smallness of the delta does not imply that it is unimportant.

Often, the delta is precisely the part that makes the new object new.

2. The Base–Delta Asymmetry Principle

The principle can be formally stated as follows:

Base–Delta Asymmetry Principle

In a structurally inherited representation, the semantic, logical, identity-related, or runtime importance of a delta is not proportional to its geometric magnitude relative to the inherited base.

Let:

```
[  
O_1 = B  
]
```

and:

```
[
```

$$O_2 = B + \Delta$$

]

A metric system may evaluate the importance of (Δ) approximately through its contribution to distance:

$$I_{\{\text{metric}\}}(\Delta)$$

f\left(

$$\frac{|\Delta|}{|B|}$$

\right)

]

A structural runtime must instead evaluate:

$$I_{\{\text{structural}\}}(\Delta)$$

g(

\text{delta type},

\text{role},

\text{scope},

\text{identity effect},

\text{runtime consequence}

)

]

In general:

[

$$I_{\{\text{metric}\}}(\Delta)$$

\neq

$$I_{\{\text{structural}\}}(\Delta)$$

]

A delta may satisfy:

[

$$\frac{|\Delta|}{|B|}$$

\ll 1

]

while simultaneously satisfying:

[

$$I_{\{\text{structural}\}}(\Delta)$$

$$\gg I_{\{\text{structural}\}}(B_i)$$

]

for many individual components (B_i) of the base.

The base may dominate the geometry.

The delta may dominate the decision.

That is the asymmetry.

3. Why Cosine Similarity Dilutes Small Deltas

Cosine similarity between vectors (x) and (y) is:

$$\frac{x \cdot y}{|x||y|}$$

Let:

$$y = x + \delta$$

Then:

$$\frac{|x|^2 + x \cdot \delta}{|x||x + \delta|}$$

To expose the key behavior, decompose the delta into components parallel and orthogonal to the base:

$$\delta = \delta_{\parallel} + \delta_{\perp}$$

where:

$$\delta_{\parallel} \parallel x$$

and:

$$x \cdot \delta_{\perp} = 0$$

The parallel component mainly changes magnitude.

Cosine similarity is largely insensitive to positive magnitude changes.

The orthogonal component changes direction, but when it is small relative to the base, its effect is also small.

For a sufficiently small orthogonal delta:

$$\cos(x, x + \delta_{\perp}) \approx 1 -$$

$$\frac{|\delta_{\perp}|^2}{2|x|^2}$$

The important consequence is that the similarity loss is approximately quadratic in the relative delta size:

$$1 - \cos(x, x + \delta_{\perp}) \approx \frac{|\delta_{\perp}|^2}{2|x|^2}$$

If:

$$\frac{|\delta_{\perp}|}{|x|}$$

then:

$$\cos(x, x + \delta_{\perp}) \approx 1$$

Even a clearly detectable structural marker may make almost no difference to the global score if it is embedded inside a large inherited representation.

This is not a computational bug.

It is the expected behavior of global directional comparison.

The problem arises when global directional comparison is treated as a complete measure of structural consequence.

4. Base Domination

The large inherited base creates what may be called **Base Domination**.

Suppose:

$$B = b_1 + b_2 + \dots + b_n$$

and the new object is:

$$O' = B + \Delta$$

As the base becomes richer:

[

$|B| \uparrow$

]

the relative geometric contribution of a fixed-size delta decreases:

[

$\frac{|\Delta|}{|B|}$

$\downarrow$

]

Therefore:

[

$\cos(B, B + \Delta)$

$\rightarrow 1$

]

as the base grows, assuming the delta remains bounded.

This creates a paradox:

The more knowledge the representation already contains, the easier it may become for one decisive new marker to disappear inside the aggregate geometry.

A mature representation may include:

- visual features;
- semantic attributes;
- contextual associations;
- historical traces;
- task relations;
- linguistic descriptions;
- prototype links;
- neighboring concepts.

A small permission marker or negation feature may then occupy only a tiny part of the total vector.

The larger and richer the base, the more severe the dilution can become.

This means the Base–Delta problem may grow with system maturity rather than disappear.

5. The Delta May Be the Primary Information

In inherited structures, most of the base is already known.

Therefore, the most informative part of the new object may be the delta.

Suppose the system already knows:

[
B
]

When it observes:

[
B+ Δ
]

the repeated base contributes little new information.

The novelty is concentrated in:

[
 Δ
]

From an information-update perspective:

[
 $\operatorname{NewInformation}(B+\Delta \mid B)$
 $\approx$
 $\operatorname{Information}(\Delta)$
]

The global representation may be dominated by the base, but the learning event is dominated by the delta.

This creates a second asymmetry:

[
 $\boxed{\text{Representational Mass} \neq \text{Update Information}}$
]

The base carries most of the stored content.

The delta carries most of the current news.

An intelligent update mechanism should therefore ask:

What changed relative to the known base?

not only:

What complete stored object is this input closest to?

This observation directly connects SRMS to:

- Differential Trees;

- Delta Algorithms;
- structural updates;
- Gap detection;
- Runtime Invariant migration;
- Information Job Shop processing.

6. Small Delta, Categorical Change

Some deltas create gradual variation.

Others create categorical transitions.

Consider:

cat

cat + black fur

The delta may describe an ordinary attribute variation.

Now consider:

execute

execute + NOT

The delta reverses the operation.

Or:

A calls B

A calls B + reverse direction

The delta changes control flow.

Or:

user authorized

user authorized + revocation marker

The delta changes admissibility.

These deltas may be similar in geometric size but radically different in structural role.

Therefore delta importance cannot be estimated solely by magnitude:

$$\left[\begin{array}{l} |\Delta_1| \\ \approx \\ |\Delta_2| \end{array} \right]$$

does not imply:

```
[
  I_{\text{structural}}(\Delta_1)
\approx
  I_{\text{structural}}(\Delta_2)
]
```

One delta may be descriptive.

Another may be constitutive.

One adds detail.

Another changes identity.

One adjusts ranking.

Another closes the gate.

This distinction requires **delta typing**.

7. A Taxonomy of Structural Deltas

The Base–Delta Asymmetry Principle becomes operational only when deltas are classified by structural function.

A preliminary taxonomy follows.

7.1 Descriptive Delta

Adds a non-decisive attribute.

Examples:

- color;

- texture;

- style;

- approximate size;

- optional contextual detail.

Typical consequence:

```
[
  \text{same structural identity}
+
  \text{refined description}
]
```

These deltas are often suitable for soft metric treatment.

7.2 Branching Delta

Creates a meaningful subtype or new differential-tree branch.

Examples:

- domestic cat versus wild cat;
- read operation versus write operation;
- local task versus distributed task.

Typical consequence:

```
[  
\text{shared parent}  
+  
\text{new branch}  
]
```

These deltas may require explicit branch registration.

7.3 Identity Delta

Creates or distinguishes an instance.

Examples:

- another cat;
- another execution event;
- another document version;
- another runtime process.

Typical consequence:

```
[  
\text{same type}  
\neq  
\text{same identity}  
]
```

These deltas must not be averaged away.

7.4 Cardinality Delta

Changes count, multiplicity, repetition, or accumulated quantity.

Examples:

- three cats to four cats;
- one vote to two votes;

- one evidence item to five evidence items;
- one edge to repeated edges.

Typical consequence:

```
[
n
\rightarrow
n+1
]
```

Cosine normalization may hide this class completely.

7.5 Role Delta

Changes the role played by an otherwise familiar component.

Examples:

- caller versus callee;
- subject versus object;
- owner versus user;
- producer versus consumer.

Typical consequence:

```
[
\text{same component}
+
\text{different slot}
\text{different structure}
]
```

7.6 Direction Delta

Reverses a relation or transition.

Examples:

- A calls B versus B calls A;
- parent-to-child versus child-to-parent;
- cause-to-effect versus effect-to-cause.

Typical consequence:

```
[  
A\xrightarrow{r}B  
\neq  
B\xrightarrow{r}A  
]
```

7.7 Scope Delta

Changes where a rule, condition, or marker applies.

Examples:

- local negation versus global negation;
- branch-specific permission versus system-wide permission;
- one-task constraint versus workflow constraint.

Typical consequence:

```
[  
\text{same marker}  
+  
\text{different scope}  
\text{different meaning}  
]
```

7.8 Logical Delta

Changes truth conditions or logical polarity.

Examples:

- true versus false;
- present versus absent;
- allowed versus forbidden;
- must versus may;
- all versus some.

Typical consequence:

```
[
```

`\text{decision boundary crossing}`
]

7.9 Trigger Delta

Changes whether an operation fires.

Examples:

- threshold reached;
- event detected;
- prerequisite satisfied;
- failure flag raised.

Typical consequence:

[
`\text{inactive}`
`\rightarrow`
`\text{active runtime path}`
]

7.10 Runtime-Invariant Delta

Changes the stable computational identity of the object.

Examples:

- same components packaged under a different operational contract;
- changed state-transition legality;
- changed capability boundary;
- changed callable interface;
- changed execution guarantee.

Typical consequence:

[
RI_1
`\rightarrow`
RI_2
]

This is among the most consequential delta classes.

8. Delta Type Matters More Than Delta Size

The preceding taxonomy suggests:

```
[
  \text{structural}(\Delta)
  g(
    T_{\Delta},
    R_{\Delta},
    S_{\Delta},
    C_{\Delta}
  )
]
```

where:

- (T_{Δ}) is the delta type;
- (R_{Δ}) is its structural role;
- (S_{Δ}) is its scope;
- (C_{Δ}) is its runtime consequence.

A useful system must distinguish:

```
[
  \text{small descriptive delta}
]
```

from:

```
[
  \text{small identity-changing delta}
]
```

and:

```
[
  \text{small runtime-changing delta}
]
```

The first may affect ranking by a small amount.

The second may require a new node.

The third may require a different execution path.

Thus:

```
[
  \boxed{
```

$\text{\text{Delta Type}}$

$\text{\text{Delta Magnitude}}$

}

]

in decisions involving structural identity and runtime consequence.

Here " $(>)$ " again denotes control priority, not universal numerical superiority.

9. The Three-Cat Illustration

The three-cat problem exposes both identity and cardinality deltas.

Suppose the system has learned a cat prototype:

[

C

]

It observes three cats:

[

$C_1, \backslash C_2, \backslash C_3$

]

Each instance may be close to the prototype:

[

$\text{\operatorname{cos}}(C_i, C) \approx 1$

]

When a fourth cat appears, the new input may be:

[

$C_4 = C + \Delta_4$

]

where (Δ_4) contains the instance-specific difference.

The system must answer two different questions.

Type question

[

$\text{\text{Is }} C_4 \text{\text{ a cat?}}$

]

Cosine similarity is well suited to this question.

Identity question

[

$\text{\text{Is }} C_4 \text{\text{ one of the previous cats?}}$

]

This requires instance recognition.

Cardinality question

[

\text{Are there now four cats?}

]

This requires persistent registration and counting.

If all instances are pooled:

[

$$X_3 = C_1 + C_2 + C_3$$

]

and:

[

$$X_4 = C_1 + C_2 + C_3 + C_4$$

]

then, under highly similar instances:

[

$$X_3 \approx 3C$$

]

and:

[

$$X_4 \approx 4C$$

]

Therefore:

[

$$\cos(X_3, X_4) \approx 1$$

]

The added cat is a small structural update relative to the total scene representation.

But it changes:

- object count;
- instance registry;
- scene state;
- possibly task consequence.

The fourth cat is therefore a decisive cardinality delta even if it is geometrically absorbed.

10. The Chinese-Character Illustration

A growing writing system cannot efficiently invent every new character as a completely unrelated holistic image.

It reuses existing visual and functional components.

A simplified generative form is:

[
Character'

KnownBase

+

SemanticMarker

+

PhoneticMarker

+

PositionalStructure

]

The inherited base may dominate visual similarity.

Yet a small radical, component, or positional difference may change:

- semantic category;
- pronunciation cue;
- lexical identity;
- morphological family;
- interpretation.

Thus two characters may be visually close while belonging to different semantic or phonetic structures.

Conversely, two visually different characters may share a common functional component.

The important information is not only the holistic picture.

It lies in:

- which components are present;
- what role each component plays;
- where each component is positioned;
- which delta distinguishes the new character from the known family.

This illustrates a general transition:

[

$$\begin{aligned}
 & \text{\text{Holistic Similarity}} \\
 & \rightarrow \\
 & \text{\text{Base Reuse}} \\
 & + \\
 & \text{\text{Typed Deltas}} \\
 & + \\
 & \text{\text{Compositional Recognition}} \\
 &]
 \end{aligned}$$

The more a representation system grows, the more important delta recognition becomes.

11. The UTN Illustration

UTN structures frequently reuse the same nodes and operations.

A new structure may differ by only one relation.

Consider:

A calls B

and:

B calls A

The shared base contains:

- A;
- B;
- call.

The delta is edge direction.

A pooled vector may represent both with nearly the same components.

Yet their runtime structures are different:

$$\begin{aligned}
 & [\\
 & A \rightarrow \text{\text{call}} B \\
 &]
 \end{aligned}$$

versus:

$$\begin{aligned}
 & [\\
 & B \rightarrow \text{\text{call}} A \\
 &]
 \end{aligned}$$

Similarly:

A calls B if condition C

differs from:

A calls B

only by a condition marker.

The added condition may be small in representation size.
 But it changes the operation from unconditional to conditional.
 Other small UTN deltas may include:

- role assignment;
- parameter binding;
- edge type;
- execution order;
- exception path;
- scope;
- authorization;
- repeat count;
- branch priority.

These examples show that UTN growth is often delta-driven.
 A system that sees only the shared base may miss the structure that makes each network operationally distinct.

12. The Asymmetry between Recognition and Reconstruction

A metric system may attempt to recognize a new object by finding the nearest complete stored representation.

This can be written as:

$$[\hat{O} \arg \max_{\{O_i\}} \cos(O', O_i)]$$

But when the object is inherited from a known base, a more informative process is:

$$[\hat{B} \operatorname{RecognizeBase}(O')]$$

]

followed by:

[

$\hat{\Delta}$

$\text{operatorname{ExtractDelta}}(O', \hat{B})$

]

and then:

[

$\text{operatorname{Interpret}}(\hat{\Delta})$

]

This changes the problem from complete-object matching to base-plus-delta analysis.

The system asks:

1. What known structure is being reused?
2. What is newly added, removed, reversed, or retyped?
3. What kind of delta is this?
4. Does it preserve identity?
5. Does it require a new branch?
6. Does it change runtime behavior?

This is more aligned with how inherited structures evolve.

13. Base Recognition and Delta Recognition Are Different Tasks

The base and the delta may require different algorithms.

Base recognition

The base is often:

- large;
- redundant;
- noisy;
- continuous;

- prototype-like;
- suitable for metric comparison.

Cosine similarity is strong here.

Delta recognition

The delta is often:

- small;
- sparse;
- typed;
- relational;
- position-sensitive;
- logically decisive;
- runtime-sensitive.

Explicit recognition may be stronger here.

This suggests a hybrid decomposition:

```
[
\boxed{
\text{Metric Base Recognition}
+
\text{Structural Delta Recognition}
}
]
```

The system may use cosine similarity to identify the inherited family:

```
[
\hat{B}
\arg\max_{B_i}
\operatorname{cos}(O',B_i)
]
```

Then it performs structural differencing:

```
[
\Delta
O'-\hat{B}
]
```

]

not necessarily as raw vector subtraction, but as a structured comparison of:

- components;
- tags;
- roles;
- edges;
- counts;
- scope;
- triggers;
- identities.

This can be understood as a metric-to-structural handoff.

14. Delta Channels Should Be Protected

One possible response to Base–Delta Asymmetry is to assign the delta a larger numerical weight.

For example:

[
 $O' = B + \lambda \Delta$
]

with:

[
 $\lambda > 1$
]

But simple reweighting is not a complete solution.

The correct value of (λ) depends on delta type.

A color delta and a negation delta should not necessarily receive the same amplification.

Furthermore, any finite weight may still be overwhelmed by a sufficiently large base.

Therefore decisive deltas should often be represented through protected channels rather than only through larger vector coefficients.

A representation may separate:

```
[
O=
(
V_{\text{base}},
D_{\text{identity}},
D_{\text{cardinality}},
D_{\text{role}},
D_{\text{logic}},
D_{\text{scope}},
D_{\text{runtime}}
)
]
```

The base vector may be scored continuously.

The delta channels may be handled through:

- exact matching;
- typed constraints;
- hard gates;
- dedicated submetrics;
- relation checks;
- state-transition checks;
- Runtime Invariant validation.

This prevents structurally decisive information from being normalized into insignificance.

15. Why a Single Weighted Score May Still Fail

A common hybrid approach is to combine many signals:

```
[
S=
w_cS_{\text{cos}}
+
w_iS_{\text{identity}}
+
w_rS_{\text{role}}
```


$$\begin{aligned}
 &+ \\
 &w_{nS_{\{\text{number}\}}} \\
 &+ \\
 &w_{sS_{\{\text{scope}\}}} \\
 &]
 \end{aligned}$$

This is better than cosine alone.

However, a simple weighted sum still allows compensation.

A high cosine score may outweigh:

- a failed role match;
- a denied permission;
- a count mismatch;
- a negation conflict.

For example:

$$\begin{aligned}
 &[\\
 &0.95S_{\{\text{cos}\}} \\
 &+ \\
 &0.05S_{\{\text{permission}\}} \\
 &]
 \end{aligned}$$

may still yield a high score even when permission is denied.

But permission denial should not merely reduce confidence.

It should reject the action.

Therefore some delta channels require non-compensatory logic:

$$\begin{aligned}
 &[\\
 &\text{\text{PermissionMismatch}} \\
 &\text{\text{Rightarrow}} \\
 &\text{\text{Reject}} \\
 &] \\
 &[\\
 &\text{\text{IdentityMismatch}} \\
 &\text{\text{Rightarrow}} \\
 &\text{\text{DoNotMerge}} \\
 &] \\
 &[\\
 &\text{\text{RoleConflict}} \\
 &\text{\text{Rightarrow}} \\
 &\text{\text{DifferentStructure}} \\
 &]
 \end{aligned}$$

[
 $\text{RuntimeInvariantConflict}$
 $\rightarrow$
 DifferentRoute
]

This is why SRMS emphasizes:

[
 $\boxed{\text{Hard Structural Gates}}$
 +
 $\text{Soft Metric Ranking}$
]

rather than merely:

[
 $\text{More Features in One Score}$
]

16. Delta-First Interpretation

When a base is already known with high confidence, the system should shift attention toward the delta.

This can be expressed as:

[
 $P(B \mid O') \approx 1$
]

Once the base is recognized, the main uncertainty becomes:

[
 $P(T_{\{\Delta\}}, R_{\{\Delta\}}, C_{\{\Delta\}} \mid O', B)$
]

where:

- $(T_{\{\Delta\}})$ is the delta type;
- $(R_{\{\Delta\}})$ is the delta role;
- $(C_{\{\Delta\}})$ is the delta consequence.

The runtime should therefore change mode:

Unknown object mode

↓

Find the likely base



Known base mode



Detect and interpret the delta

This resembles differential diagnosis.

The first task identifies the general family.

The second identifies the small difference that determines the actual case.

A system that continues treating the whole object as equally uncertain may waste attention on the already-known base and underweight the decisive delta.

17. The Delta as a Structural Event

A delta is not always merely a static feature.

It may be an event.

Examples:

- one new cat enters the scene;
- a permission is revoked;
- an edge is reversed;
- a threshold is crossed;
- a task becomes mandatory;
- a local rule becomes global;
- a Runtime Invariant is updated.

In these cases:

[
\Delta
\text{state transition}
]

The system should represent:

[
S_t
→ \Delta
S_{t+1}
]

The importance of the delta lies in the transition, not only in the final vector. This connects the Base–Delta Asymmetry Principle with State–Operation–State reasoning:

```
[  
  \text{State}  
  \rightarrow  
  \text{Operation or Event}  
  \rightarrow  
  \text{New State}  
]
```

A final-state similarity score may miss the significance of how the state changed.

Therefore delta-aware systems should preserve:

- previous state;
- new state;
- transition type;
- trigger;
- causal path;
- update consequence.

18. Base–Delta Asymmetry and Differential Trees

Differential Trees are natural structures for representing inherited bases and branching deltas.

A parent node represents a shared base:

```
[  
  B  
]
```

Child nodes represent:

```
[  
  B+\Delta_1  
]  
[  
  B+\Delta_2  
]
```

[
 $B + \Delta_3$
]

However, if branch routing depends only on global cosine similarity, small deltas may become crowded near the parent.

As the number of children grows:

[
 $\cos(B + \Delta_i, B + \Delta_j)$
 $\rightarrow 1$
]

for small (Δ_i) and (Δ_j) .

This can produce:

- narrow margins;
- unstable ranking;
- branch collisions;
- sensitivity to noise;
- incorrect merging;
- ambiguous updates.

Typed deltas can strengthen the tree.

A node may register:

Parent Base

- |—— Descriptive Delta Branches
- |—— Identity Delta Registry
- |—— Cardinality Updates
- |—— Role Delta Branches
- |—— Logical Delta Gates
- |—— Scope Delta Branches
- |—— Runtime-Invariant Transitions

The tree then becomes both:

- a metric neighborhood;
- a structural delta registry.

19. Base–Delta Asymmetry and Runtime Invariants

A Runtime Invariant may preserve a stable operational identity across representational variation.

Ordinary descriptive deltas may leave the RI unchanged:

```
[  
RI(B)=RI(B+\Delta_d)  
]
```

A decisive runtime delta may create a different invariant:

```
[  
RI(B)\neq RI(B+\Delta_r)  
]
```

The challenge is that:

```
[  
|\Delta_d|  
]
```

may be larger than:

```
[  
|\Delta_r|  
]
```

even though ($|\Delta_r|$) is more important.

For example, a major stylistic rewrite may preserve the same runtime behavior.

A one-bit authorization change may alter whether the operation is legal.

Thus RI transition cannot be inferred solely from geometric delta size.

The system needs a function:

```
[  
\operatorname{RIDeltaClassify}(B,\Delta)  
]
```

which returns:

RI-preserving

RI-refining

RI-branching

RI-invalidating

RI-replacing

This provides a direct bridge from SRMS to Runtime Invariant Architecture.

20. Base–Delta Asymmetry and Structural Feasibility

A small delta may determine whether a proposed structure is feasible.

Suppose a large plan (B) is structurally sound.

Adding one constraint:

[
B+\Delta
]

may make the plan infeasible.

Examples:

- one missing interface;
- one incompatible type;
- one denied permission;
- one impossible deadline;
- one broken dependency;
- one violated invariant.

The majority of the structure remains unchanged.

Cosine similarity between the feasible and infeasible versions may be extremely high.

But feasibility is discontinuous:

[
F(B)=1
]
[
F(B+\Delta)=0
]

This leads to another important expression:

[
\boxed{
\text{Small Geometric Change}
\rightarrow
\text{Large Feasibility Change}
}
]

Therefore feasibility-critical deltas must be recognized before similarity-based commitment.

21. Base–Delta Asymmetry and Information Job Shops

Within an Information Job Shop, information pieces may arrive as updates to

existing structural assets.

The incoming information job is often not the full object.

It is a delta:

- add an edge;
- change a role;
- update a count;
- revoke a permission;
- create a new instance;
- close a branch;
- trigger a task;
- modify a Runtime Invariant.

The information value of the job depends on the delta's effect on the current system state.

Thus:

```
[  
\operatorname{JobValue}(\Delta)  
\neq  
|\Delta|  
]
```

A tiny information job may cause major reconfiguration.

A large descriptive input may cause almost none.

The IJS runtime should therefore schedule and prioritize deltas by:

- structural novelty;
- dependency impact;
- runtime consequence;
- feasibility effect;
- branch-creation potential;
- RI transition.

This is another manifestation of Base–Delta Asymmetry.

22. Recognition-Gated Cosine Scoring under Base–Delta Asymmetry

The Base–Delta Asymmetry Principle provides the theoretical basis for RG-CST.

An RG-CST may process an object in five stages.

Stage 1 — Base Recognition

Use metric similarity to identify the most plausible inherited base:

$$\left[\begin{array}{l} \hat{B} \\ \arg\max_{B_i} \cos(O', B_i) \end{array} \right]$$

Stage 2 — Structural Delta Extraction

Identify differences between the input and the recognized base:

$$\left[\begin{array}{l} \hat{\Delta} \\ \text{StructuralDiff}(O', \hat{B}) \end{array} \right]$$

Stage 3 — Delta Typing

Classify the delta:

$$\left[\begin{array}{l} T_{\Delta} \\ \text{Classify}(\hat{\Delta}) \end{array} \right]$$

Possible classes include:

- descriptive;
- identity;
- cardinality;
- role;
- direction;
- scope;

- logical;
- trigger;
- Runtime-Invariant-changing.

Stage 4 — Recognition Gating

Apply delta-specific gates:

```
[
G( $T_{\Delta}$ ,  $\hat{\Delta}$ )
]
```

Examples:

- new identity → do not merge;
- permission denial → reject execution;
- role reversal → select different branch;
- count increment → update registry;
- RI change → route to a different runtime node.

Stage 5 — Local Metric Ranking

Only after structural admissibility is established does cosine rank candidates within the valid branch.

This is not merely a patch to cosine similarity.

It is a division of labor:

```
[
\boxed{
\text{Cosine Finds the Base}
}
]
[
\boxed{
\text{Recognition Interprets the Delta}
}
]
```

23. A Proposed Delta Registry

Each RG-CST node may maintain a Delta Registry.

A conceptual form is:

Delta Registry

- |—— Known Descriptive Deltas
- |—— Identity-Creating Deltas
- |—— Cardinality Updates
- |—— Role Reassignments
- |—— Direction Reversals
- |—— Scope Changes
- |—— Logical Operators
- |—— Trigger Conditions
- |—— Forbidden Deltas
- |—— Runtime-Invariant Transitions

A registry entry may include:

Delta ID

Delta Type

Affected Component

Structural Role

Scope

Admissibility Rule

Merge Policy

Branch Policy

Runtime Consequence

Confidence

Evidence

This allows the tree to learn not only prototypes, but also the ways in which prototypes may legally and meaningfully change.

A mature structural intelligence system therefore needs both:

[
 \text{Prototype Memory}
]

and:

[
 \text{Delta Memory}
]

24. Failure Modes Predicted by the Principle

The Base-Delta Asymmetry Principle predicts several recurring failure modes.

24.1 Delta Dilution

The decisive marker contributes too little to the global metric.

24.2 Base Overconfidence

Strong recognition of the shared base causes the system to ignore the new difference.

24.3 False Merge

A new identity or branch is merged into an existing object.

24.4 False Equivalence

Two role-reversed or scope-different structures are treated as equivalent.

24.5 Runtime Misrouting

A small trigger or permission delta fails to redirect execution.

24.6 Cardinality Loss

Repeated instances collapse into one normalized direction.

24.7 Branch Crowding

Many delta-derived children occupy nearly the same cosine neighborhood.

24.8 Update Blindness

The system repeatedly recognizes what remains the same but fails to isolate what changed.

These failure modes provide a concrete experimental agenda for SRMS.

25. Experimental Questions

The principle can be investigated through controlled tests.

Experiment 1 — Delta-to-Base Ratio

Construct:

[
 $O' = B + \Delta$
]

while increasing:

[
 $|B|$
]

and holding:

[
 $|\Delta|$
]

constant.

Measure how quickly cosine sensitivity to the delta declines.

Experiment 2 — Same-Size, Different-Role Deltas

Create deltas of similar geometric magnitude but different structural types:

- color change;
- identity marker;
- negation;
- permission denial;
- edge reversal.

Test whether a pure metric model distinguishes their consequences.

Experiment 3 — Cardinality Preservation

Compare representations of:

- one instance;
- two instances;
- three instances;
- repeated identical instances.

Determine when cosine normalization collapses count information.

Experiment 4 — Role Reversal

Compare:

[
 $A \xrightarrow{r} B$
]

with:

[
 $B \xrightarrow{r} A$
]

under different pooling and graph-encoding methods.

Experiment 5 — Branch Crowding

Add many small-delta children to the same base and measure:

- pairwise cosine similarity;
- nearest-neighbor stability;

- margin reduction;
- routing error.

Experiment 6 — Recognition Gating

Compare:

- pure cosine ranking;
- weighted hybrid scoring;
- hard-gated cosine ranking;
- RG-CST with typed deltas.

Evaluate identity, count, role, scope, and runtime accuracy.

26. Design Rules Derived from the Principle

The Base-Delta Asymmetry Principle supports several engineering rules.

Rule 1 — Do Not Infer Importance from Magnitude Alone

Small representational differences may be structurally decisive.

Rule 2 — Separate Base Recognition from Delta Recognition

Use the algorithm best suited to each task.

Rule 3 — Preserve Non-Compensatory Deltas

Identity, negation, permission, role, and RI changes should not be treated as ordinary weighted features.

Rule 4 — Protect Cardinality

Do not rely on normalized direction when quantity carries meaning.

Rule 5 — Register Delta Types

A tree should know not only how objects differ, but what kind of difference is present.

Rule 6 — Route by Consequence

Delta interpretation should determine merge, branch, reject, update, or trigger behavior.

Rule 7 — Reallocate Attention after Base Recognition

Once the base is known, prioritize the novel delta.

Rule 8 — Preserve Transition History

For event-like deltas, store state change rather than only the final state.

27. Relation to Minimal Evolution Threshold

The Base–Delta pattern is not accidental.

It is often produced by MET.

An evolving system tends to preserve working structure and add the minimum sufficient modification:

$$\begin{bmatrix} S_{t+1} \\ S_t + \Delta_{\min} \end{bmatrix}$$

where $(\Delta_{\min})$ is the smallest delta that achieves the required new function.

This means successful evolution naturally creates:

- large inherited bases;
- small functional deltas;
- increasing structural reuse;
- growing dependence on delta interpretation.

Therefore MET-driven systems may systematically generate conditions under which pure global similarity becomes less reliable.

Paradoxically:

The more efficient the evolutionary reuse, the more important explicit delta recognition becomes.

A system that repeatedly follows MET will accumulate many descendants that share most of their structure.

The intelligence challenge shifts from recognizing the common base to interpreting the small differences.

28. Relation to the Metric-to-Compositional Transition

Small systems may represent objects holistically.

As the system expands, complete independent representations become inefficient.

It begins to reuse:

- bases;
- components;

- roles;
- markers;
- relations;
- templates;
- Runtime Invariants.

The representation therefore moves toward:

```
[
\text{Base}
+
\text{Typed Delta}
]
```

This is part of a broader transition:

```
[
\boxed{
\text{Holistic Metric Recognition}
\rightarrow
\text{Compositional Structural Recognition}
}
]
```

Metric similarity remains useful for identifying families and prototypes. Compositional recognition becomes necessary for interpreting how reused structures differ.

Base–Delta Asymmetry is one of the forces driving this transition.

29. Core Propositions

This paper establishes the following propositions.

Proposition 1 — Base Domination

For a bounded delta (Δ), increasing the magnitude or dimensional richness of the shared base (B) tends to increase the cosine similarity between (B) and $(B+\Delta)$.

Proposition 2 — Delta Importance Independence

The structural importance of a delta cannot be inferred from its geometric magnitude alone.

Proposition 3 — Novelty Concentration

When the base is already known, most new information may be concentrated

in the delta rather than the complete representation.

Proposition 4 — Typed Consequence

Deltas of similar geometric size may require entirely different runtime actions.

Proposition 5 — Non-Compensatory Structure

Some deltas must be represented as gates or constraints rather than ordinary weighted features.

Proposition 6 — Base–Delta Algorithmic Separation

Base recognition and delta interpretation may require different representations and scoring mechanisms.

Proposition 7 — Growth Amplification

As a system grows through structural reuse, Base–Delta Asymmetry may become more severe because descendant representations share increasingly large bases.

30. Key Takeaways

1. New structural objects are often created through inheritance:

[
O'=B+\Delta
]

2. The base may dominate the representation, while the delta dominates the decision.
3. Cosine similarity naturally emphasizes the shared geometric direction and may dilute a small delta.
4. A small delta may change identity, cardinality, role, scope, logical polarity, permission, trigger state, or Runtime Invariant.
5. Geometric delta size and structural delta importance are different quantities.
6. Once a base is recognized, attention should shift toward what changed.
7. Deltas must be typed by structural function, not only measured by vector magnitude.
8. Hard structural gates are required for non-compensatory differences.

9. Recognition-Gated Cosine Scoring Trees can use cosine similarity for base recognition while using explicit structural recognition for delta interpretation.
10. MET-driven structural reuse makes delta recognition increasingly important as intelligent systems mature.

31. Closing Perspective

The central difficulty examined by SRMS does not arise because cosine similarity is weak.

It arises because inherited structures are efficient.

When a system reuses a large working base and adds only the minimum sufficient change, the new meaning becomes concentrated in a small delta.

The common base answers:

What kind of structure is this?

The delta answers:

What is different now?

Is this a new instance?

Has the count changed?

Has the relation reversed?

Has permission been denied?

Has the Runtime Invariant changed?

A metric system may recognize the base with great confidence while still missing the decisive update.

Therefore intelligent recognition must not stop at:

```
[
\operatorname{cos}(B,B+\Delta)\approx 1
]
```

It must continue to ask:

```
[
\boxed{
\text{What is }\Delta\text{, and what does it do?}
}
]
```

That question is the essence of the Base–Delta Asymmetry Principle and one of the primary theoretical foundations of Structural Recognition above Metric Similarity.